

아키텍처 설계와 클래스 설계



학습개요

본 주차의 학습목표와 학습내용을 확인하세요.

학습목표

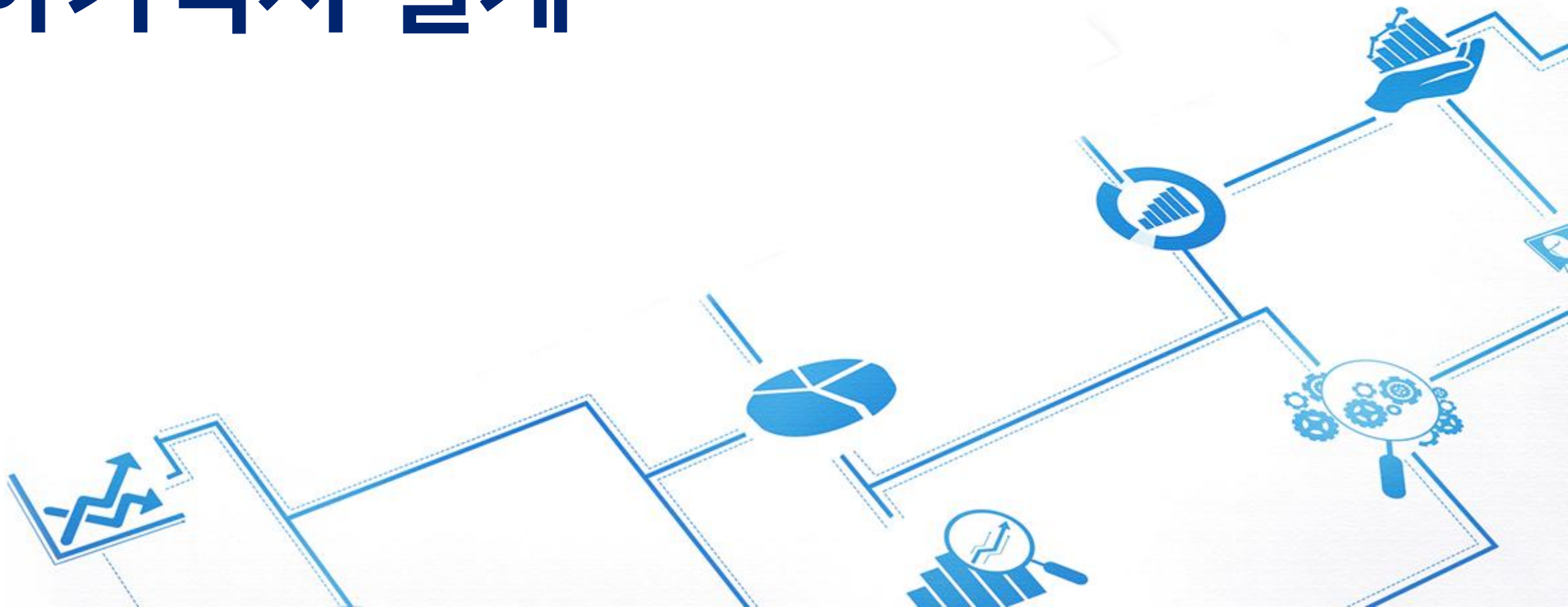
- ☑ 아키텍처의 품질 속성을 살펴본다.
- ☑ 아키텍처 스타일의 종류를 알아본다.
- ☑ 클래스 간의 관계를 알아본다.
- ☑ 클래스 설계 원칙의 종류를 알아본다.

학습내용

- 1 아키텍처 설계
- 2 클래스 간의 관계
- 3 클래스 설계 원칙



아키텍처 설계



아키텍처 설계

- 아키텍처
 - 건물의 뼈대뿐 아니라 특성을 결정짓는 기본 구조를 일컫는 말
 - 아키텍처는 모든 기술 분야에 적용할 수 있고 종류도 다양함



그림 6-1 건물 지을 때의 아키텍처

아키텍처 설계

- 아키텍처의 필요성

- 복잡성의 문제

- 대형 프로젝트는 전체 시스템의 구조를 생각하며 균형과 조화를 이루도록 설계
 - 대형 프로젝트가 복잡성의 문제를 해결하는 방법
 - 개발할 소프트웨어의 전체 구조를 가장 먼저 생각
 - 소프트웨어의 구조를 이루는 각 구성 요소를 찾을
 - 각 구성 요소 간의 명확한 관계를 설정
 - 일정한 규칙을 따름

- 아키텍처의 필요성

- 복잡하고 규모가 큰 소프트웨어를 개발하려면 전체적인 구조가 유기적으로 잘 구성되어야 함
 - 잘 정의된 구조의 품질 좋은 소프트웨어를 만 들려면 소프트웨어 아키텍처가 필요
 - 아키텍처 설계로 소프트웨어가 어떤 구조이고 어떻게 동작할 것인지를 예측할 수 있으며, 변경에 유연하게 대처 가능

아키텍처 설계

- 아키텍처의 특징과 기능

- 아키텍처의 특징

- 소프트웨어의 골격을 나타내는 추상화된 전체 구조를 제공
 - 소프트웨어를 이루고 있는 여러 구성 요소(서브시스템, 컴포넌트)를 다룸
 - 인터페이스를 통해 소프트웨어의 구성 요소가 어떻게 상호작용하는지를 정의
 - 세부 내용보다는 중요 내용(설계자가 주관적으로 판단하고 결정)만 다룸
 - 설계에 적용되는 원칙과 지침이 있음

- 아키텍처 설계 시 고려 사항

- 모든 이해관계자 사이의 의사소통 도구로 활용할 수 있어야 함
 - 구현에 대한 제약 사항(개발 비용, 기간, 조직의 역량 등)을 정의해야 함
 - 모든 이해관계자의 품질 요구사항을 반영해 우선순위에 따라 시스템 품질 속성을 결정해야 함
 - 성능, 사용성, 보안성, 안정성, 검증 가능성, 변경 용이성 등
 - 특정 문제 영역에 적합한 소프트웨어의 구성 요소를 표준화하고 패턴화해 재사용할 수 있도록 설계해야 함

아키텍처 설계

- 아키텍처의 특징과 기능

- 아키텍처의 기대 효과

- 소프트웨어의 기본 골격이 만들어져 개발에 참여하는 사람들의 이해의 폭이 넓어지며 구현상의 문제점을 도출할 수 있음
 - 소프트웨어 아키텍처를 기반으로 분할 방법을 찾고 구조화를 위한 구체적인 방안을 생각할 수 있음
 - 설계의 원칙과 가이드를 제공할 수 있고 설계를 재사용할 수 있음
 - 소프트웨어 아키텍처를 기준으로 개발 조직을 만들 수 있음
 - 전사 조직을 소프트웨어 아키텍처에 맞게 재편할 수 있음
 - 품질 특성에 대한 평가 방법을 결정할 수 있음

아키텍처의 품질 속성

- **품질 속성의 개요**

- 사용성, 이식성, 유지보수 용이성과 같이 이해관계자의 요구사항과 관심사를 반영한 것
- 품질 요구사항: 이해관계자들이 아키텍처에 요구하는 수준의 품질 속성, 가능한 정확한 수치로 제시되는 것이 좋음

- **품질 속성을 반영하는 방법**

- 해당 프로젝트에서 중요하게 생각하는 품질 속성을 결정
- 결정한 품질 속성을 어느 정도 수준으로 설계할 것인지 목표를 설정
- 설정한 목표를 달성할 수 있는 방법을 서술
- 품질 속성을 평가할 방법을 서술

- **예시) 중요한 품질 속성으로 보안성을 결정할 경우**

- 계층 구조 아키텍처를 선택하고 보호 해야 할 요소를 가장 깊은 내부 계층에 둠
- 이 계층의 데이터는 높은 등급의 보안 인증을 통과해야만 사용할 수 있도록 설계

아키텍처의 품질 속성

- 시스템 품질 속성

- 가용성

- 시스템이 실패 없이 운용될 수 있는 확률로 시스템이 장애 발생 없이 서비스를 제공할 수 있는 능력
 - 가용성이 99.99%라면 제대로 동작하지 않을 확률이 0.01%라는 의미
 - 하드웨어는 가용성을 높이기 위해 이중화 설계를 함
 - 이중화 설계: 하나의 시스템이 중단 되어도 바로 또 다른 시스템이 가동

- 변경 용이성

- 사용자가 새로운 요구사항을 요청했을 때 얼마나 쉽게 변경할 수 있는지를 말함
 - 변경 용이성을 위해서는 추적이 가능하도록 아키텍처를 설계해 변경 발생 시 영향이 미치는 곳을 쉽게 찾을 수 있어야 함
 - 변경이 자주 발생하는 소프트웨어는 아키텍처를 설계할 때 변경 용이성을 다른 품질 속성보다 우선으로 고려

아키텍처 설계

아키텍처의 품질 속성

- 시스템 품질 속성

- 성능

- 사용자 요청과 같은 이벤트가 발생했을 때 얼마나 빠르고 효율적으로 기능을 수행 할 수 있는가를 말함
 - 성능이 좋고 나쁨은 사용자 요청에 관한 결과를 제공하는 대기시간이나 초당 처리할 수 있는 트랜잭션의 수로 표현
 - 성능이 중요한 소프트웨어 라면 공유 자원을 어떻게 사용해서 설계하는지와 어떤 알고리즘을 선택하는지가 중요

아키텍처의 품질 속성

- 시스템 품질 속성
 - 보안성
 - 허용되지 않은 접근에 대응할 수 있는 능력
 - 시스템 접근의 적법성을 가려서 승인되지 않은 사용을 차단하고 올바른 사용자에게 서비스가 제공될 수 있게 설계
 - 네트워크를 통해 데이터를 주고받을 때는 보안유지를 위해 DoS 공격을 실시간으로 탐지해야 하고 회피 전략을 설립
 - 보안성 속성이 중요하다면 인증받은 일부만 접근할 수 있도록 계층 구조 아키텍처를 사용해 가장 안쪽 계층에 자료를 저장

아키텍처의 품질 속성

- 시스템 품질 속성

- 사용성

- 시스템을 사용할 때 발생할 수 있는 여러 가지 상황을 극복할 수 있도록 이를 반영해 아키텍처 설계 작업을 해야함
 - 시스템의 도움말 기능은 사용자에게 실제 도움을 줄 수 있어야 함
 - 사용자가 원치 않는 상황에 놓인 경우에도 친숙한 사용자 인터페이스를 제공해 당황하지 않도록 설계해야 함
 - 사용자의 실수에도 오류의 영향을 최소화하도록 되돌리거나 취소 기능을 제공해야 함

- 테스트 용이성

- 테스트 비용을 줄이기 위해서는 아키텍처 설계부터 테스트를 고려해야 함
 - 테스트가 얼마나 효과적으로 수행되는지, 소요되는 시간을 얼마나 단축할 수 있는지 와 같은 요소를 아키텍처 설계에 반영

아키텍처의 품질 속성

- 비즈니스 품질 속성

- 시장 적시성

- 적절한 시기에 소프트웨어를 출시해 경쟁력을 높이는 것
 - 아키텍처 설계를 할 때 구매한 컴포넌트들이 잘 맞을 수 있도록 범용성 높은 설계가 필요

- 비용과 이익

- 아키텍처를 설계할 때 비용을 많이 들여 유연한 설계를 할 것인지, 비용을 절감해 이익을 높일 것인지 판단
 - 개발 시점에서 비용을 더 들여 유연한 시스템을 만들면 나중에 수정 및 유지보수가 쉬운 시스템
 - 비용 절감에 초점을 두면 우선 개발 비용이 적게 드는 효과는 있지만, 유지보수를 할 때 비용 증가

아키텍처 설계

아키텍처의 품질 속성

- 비즈니스 품질 속성
 - 예상 시스템 수명
 - 시스템을 구축할 때는 시스템 사용 기간을 예측해 아키텍처 설계에 반영
 - 예상 시스템 수명을 위해서는 확장성, 이식성과 같은 아키텍처 품질 속성을 고려해야 함
 - 목표 시장
 - 목표 시장이 있는 소프트웨어는 경쟁 제품과 비교했을 때 기능성이 매우 중요
 - 다양한 플랫폼에서 잘 작동해야 하므로 시스템 품질 속성 중 이식성을 충분히 고려해야 함

아키텍처 설계

아키텍처의 품질 속성

- 비즈니스 품질 속성
 - 신규 발매(공개) 일정
 - 공지한 일정에 개발을 못 맞출 경우에는 현재 전에서는 기본 기능만 제공하고 차기 버전에서 기능을 추가해 완성도를 올림
 - 아키텍처 설계 시 유연성과 확장성을 중요하게 고려해야 함
 - 기존 시스템과의 통합
 - 개발할 시스템을 기존 시스템과 통합해야 한다면 기존 시스템의 특성을 잘 파악해야 함
 - 기존에 사용하던 DBMS나 개발 시 제약 사항을 충분히 고려한 아키텍처 설계가 필요

아키텍처의 품질 속성

- 아키텍처 품질 속성

- 개념적 무결성

- 시스템 설계는 전체 시스템을 나타내는 설계와 세부 구성 요소 설계로 나뉨
 - 세부 구성 요소를 전체 시스템으로 통합하더라도 일관성이 유지되어야 하기 때문
 - 전체 시스템과 시스템 구성 요소가 일관되도록 아키텍처를 결정해야 함

- 정확성과 완전성

- 정확성과 완전성은 사용자가 요구하는 기능을 충족시키는 정도를 말하며 요구분석명세 와 일치하는 정도를 나타냄

- 개발 용이성(구축 가능성)

- 전체 시스템을 적절한 모듈로 분할한 후 알맞게 분배해 개발함으로써 정해진 기간 내에 개발을 완성할 수 있고 개발 과정 중에도 쉽게 변경 할 수 있는 능력을 말함

아키텍처의 4+1 관점

• 4+1 관점

- 관점: 시스템을 이루는 요소의 집합과 그들의 연관 관계를 추상적으로 표현한 것
- 4+1 관점은 크루첸이 1995년에 쓴 논문에서 처음 등장
- 문제 영역의 1개 관점(시나리오 관점)과 해법 영역의 4개 관점(논리적 관점, 프로세스 관점, 개발 관점, 물리적 관점)으로 이루어짐

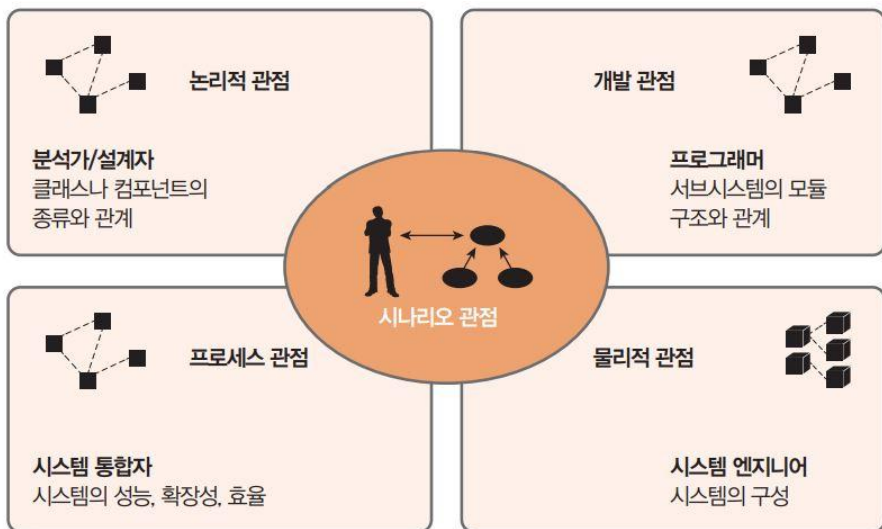


그림 6-2 UML의 4+1 관점

표 6-1 4+1 관점에 대한 크루첸과 RUP의 용어 비교

영역	관점	크루첸Kruchten	RUP
문제 영역	시나리오 관점	scenario view	usecase view
해법 영역	논리적 관점	logical view	design view
	프로세스 관점	process view	process view
	개발 관점	development view	implementation view
	물리적 관점	physical view	deployment view

아키텍처의 4+1 관점

- 시나리오(유스케이스) 관점
 - 최종 사용자가 인식하는 시스템의 기능을 의미
 - 시스템이 사용자에게 제공하는 기능에 주목하는 관점
 - 기능 하나하나가 유스케이스로 표현되기 때문에 유스케이스 관점이라 할 수 있음
 - 사용 다이어그램
 - 정적 표현: 유스케이스 다이어그램
 - 동적 표현: 상태 다이어그램, 순차 다이어그램, 통신 다이어그램, 활동 다이어그램

아키텍처의 4+1 관점

- 논리적(디자인) 관점

- 시스템의 기능에 관심이 있는 유스케이스 관점과 달리 시스템 내부를 들여다봄
- 시스템의 기능을 제공하기 위해 필요한 클래스나 컴포넌트 및 이들의 관계에 초점
- 사용 다이어그램
 - 정적 표현: 클래스 다이어그램, 객체 다이어그램
 - 동적 표현: 상태 다이어그램(클래스 내의 동작 표현), 순차·통신 다이어그램(클래스 간 상호 작용 표현), 활동 다이어그램(클래스의 연산 동작 표현)

아키텍처의 4+1 관점

- 개발(구현) 관점

- 물리적 시스템에서 사용하는 소프트웨어 서브시스템의 모듈(원시 코드, 데이터 파일, 컴포넌트, 실행 파일 등으로 구성)이 서로 어떤 연관 관계가 있고 또 설계와 어떻게 연결 관계를 나타내는지에 관심
- 사용 다이어그램
 - 정적 표현: 컴포넌트 다이어그램
 - 동적 표현: 상태 다이어그램, 순차·통신 다이어그램, 활동 다이어그램

- 물리적(배치) 관점

- 시스템에서 필요한 하드웨어 환경을 포함해 시스템을 구성하는 처리 장치 간의 물리적인 배치에 초점
- 사용 다이어그램
 - 정적 표현: 배치 다이어그램
 - 동적 표현: 상태 다이어그램, 순차 다이어그램, 통신 다이어그램, 활동 다이어그램

아키텍처 스타일

- 아키텍처 스타일

- 아키텍처 스타일은 각각의 특징이 있고 해당 스타일만의 구성과 모양을 가지고 있음

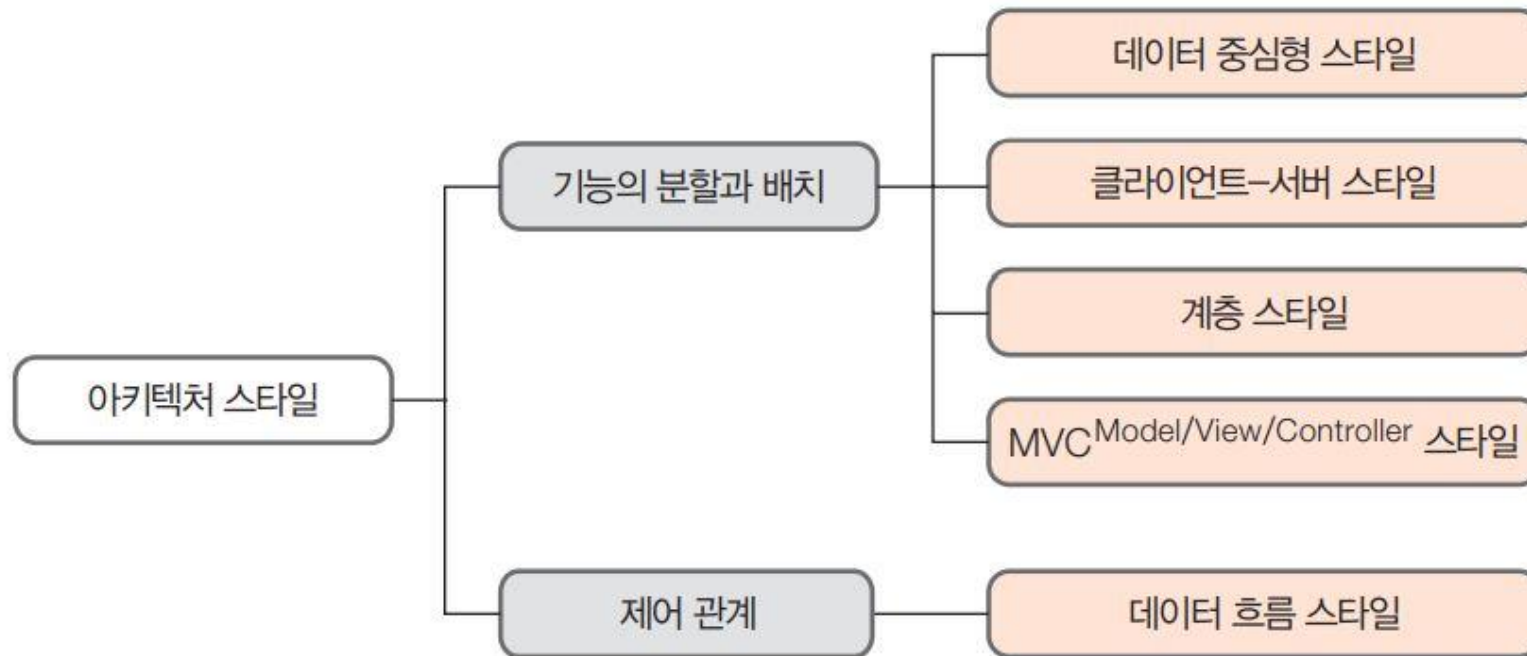


그림 6-3 아키텍처 스타일의 분류

아키텍처 스타일

- 데이터 중심형 스타일
 - 가장 큰 특징은 주요 데이터가 리포지토리에서 중앙 관리된다는 점
 - 리포지토리와 여기에 접근하는 서브시스템으로 구성
 - 시스템에서 공동으로 활용하는 데이터는 리포지토리에 보관
 - 대량의 데이터를 공유하는 은행 업무 시스템에 매우 유용
 - 서브시스템과 리포지토리의 결합도가 높아 리포지토리를 변경하면 서브시스템에 영향을 줄 수 있다는 단점

아키텍처 스타일

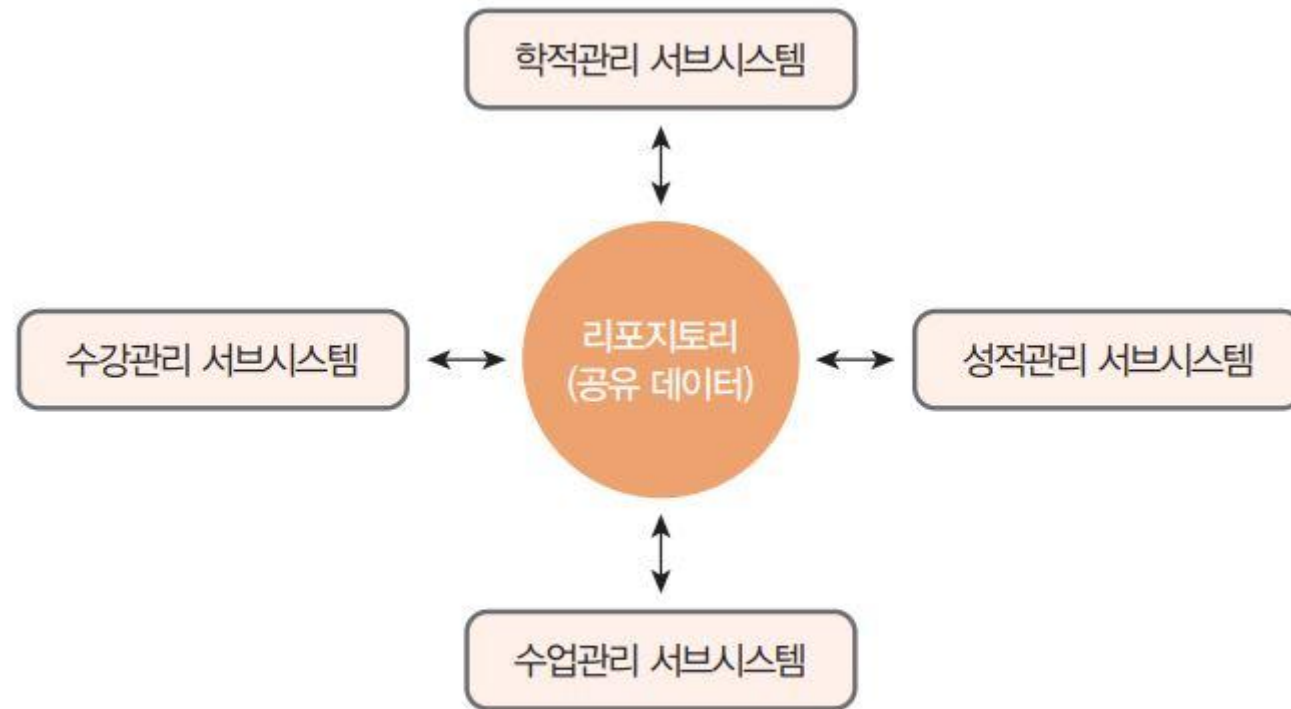


그림 6-4 데이터 중심형 스타일

아키텍처 스타일

- 클라이언트-서버 스타일

- 네트워크를 이용하는 분산 시스템 형태로 데이터와 처리 기능을 클라이언트와 서버에 분할해 사용
- 서버, 서비스, 클라이언트로 구성되며 서브시스템(컴포넌트)이 서비스를 서로 요청하면서 상호작용

- 서버 특성

- 클라이언트(서브시스템)에 서비스를 제공하는 역할을 하므로 클라이언트의 요청을 처리하기 위해 대기
- 요청을 처리한 후에는 결과를 클라이언트에 회신
- 프린트 서버, 파일 및 FTP 서버, 메일 서버, DNS 서버 등이 이에 해당

아키텍처 스타일

• 클라이언트 특성

- 사용자와 대화하기 위해 서버가 제공하는 서비스를 요청(호출)하는 서브시스템
- 메시지나 원격 프로시저 호출을 통해 서비스를 요청하고 서버가 이 요청을 받아 수행하면 그 결과를 클라이언트에게 전송
- 클라이언트가 서버에 서비스를 요청할 때는 서버의 이름과 제공서비스를 알아야 함
- 인터넷을 통해 웹서버와 통신해 웹 페이지를 다운로드하고 사용자에게 보여주는 웹브라우저가 해당

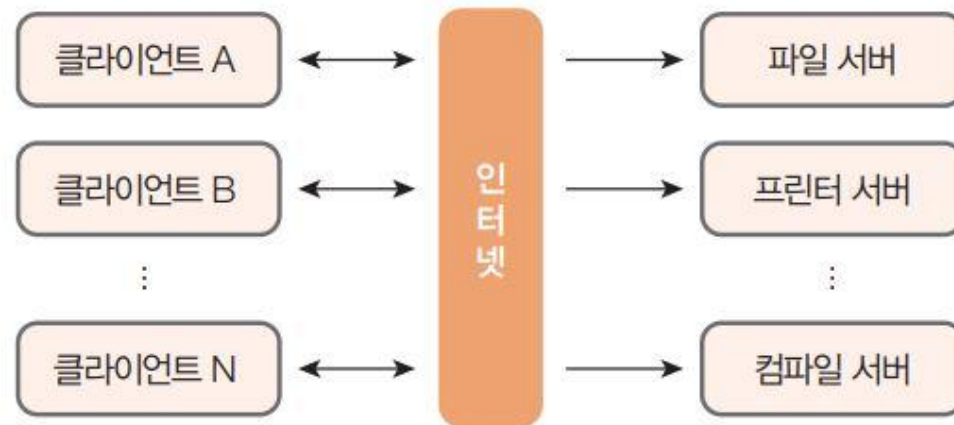


그림 6-5 클라이언트-서버 스타일

아키텍처 스타일

- 클라이언트-서버 스타일(비중의 분산)
 - 썬 클라이언트 스타일
 - 데이터 관리가 서버에서 수행되며 클라이언트는 프레젠테이션 계층만 구현
 - 데이터 관리가 거의 필요 없는 애플리케이션이나 응용 처리가 거의 필요 없는 데이터 중심의 애플리케이션의 경우에 적합
 - 서버와 네트워크에 걸리는 부하가 큰 것이 단점
 - 팻 클라이언트 스타일
 - 서버에서 데이터 관리만 관여하고 애플리케이션로직 과 프레젠테이션의 많은 부분이 클라이언트 쪽에서 구현
 - 클라이언트에 더 많은 부하가 걸릴 수밖에 없으므로 클라이언트 컴퓨터의 사양이 충분할 경우에 사용
 - 썬 클라이언트 스타일보다 관리하기가 복잡하고 새로운 버전이 출시되면 모든 클라이언트에 배포 및 설치해야 함

아키텍처 스타일

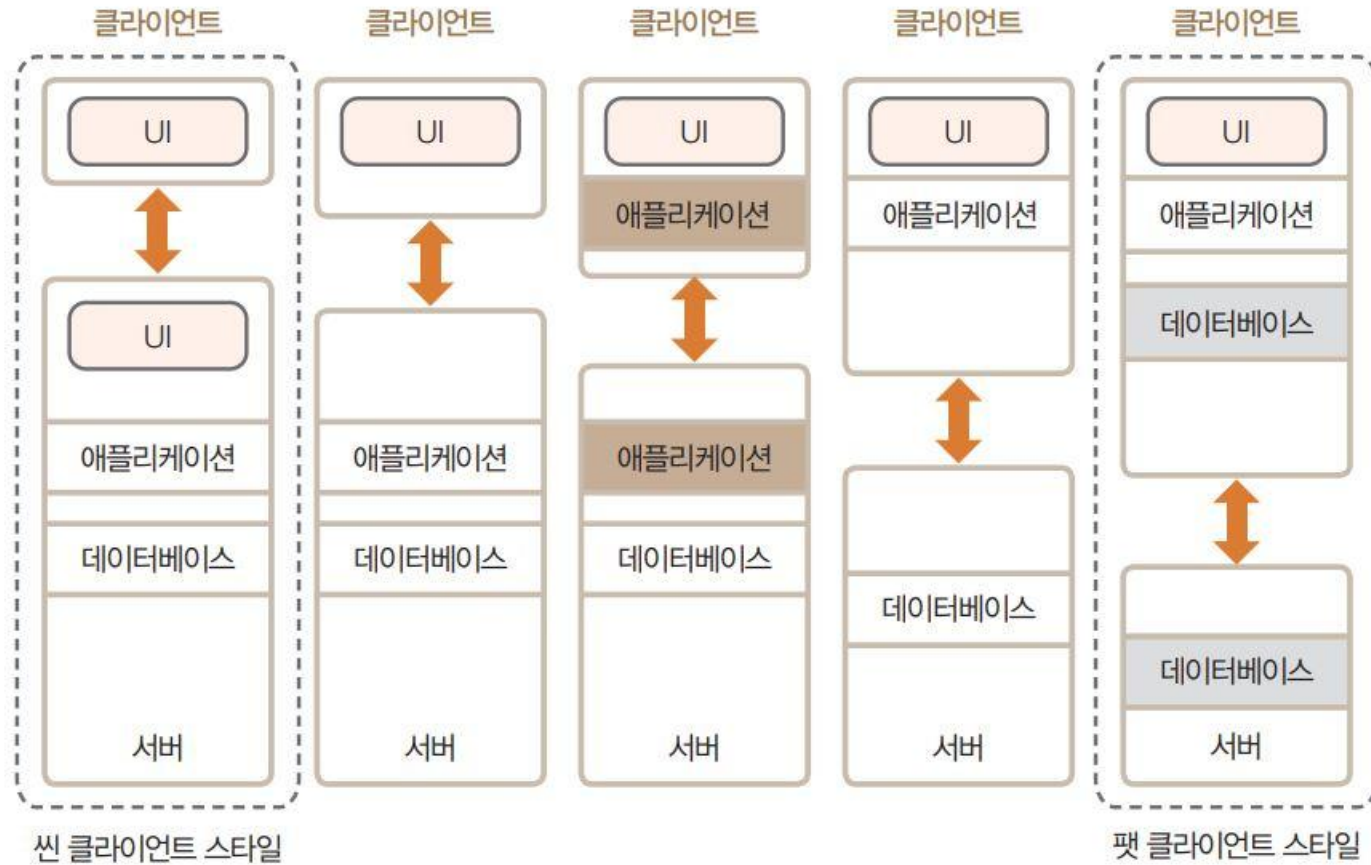


그림 6-6 비중에 따른 클라이언트 서버 스타일

아키텍처 스타일

- 계층 스타일
 - 기능을 몇 개의 계층으로 나누어 배치
 - 데이터베이스를 많이 이용하는 소프트웨어는 3-계층으로 구성
 - 각 계층의 응집도는 높고 계층 간 결합도는 낮아 재사용 및 유지보수가 용이한 것이 장점
 - 계층 간 역할 분담이 명확해 각 계층을 쉽게 변경할 수 있음
 - 대부분의 운영체제와 통신 시스템이 계층 스타일을 이용

아키텍처 스타일

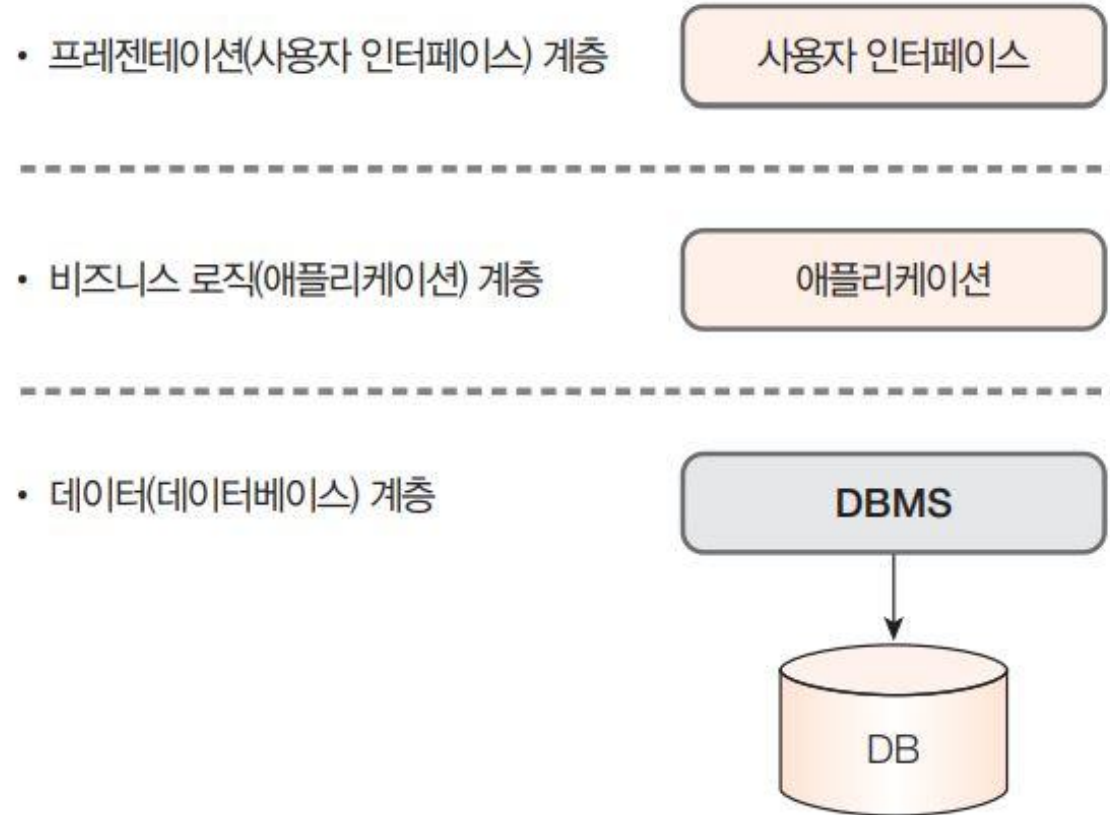


그림 6-7 3-계층 스타일

아키텍처 스타일

- 계층 스타일
 - 프레젠테이션
 - 클라이언트 계층으로서 사용자와 직접 만나는 화면에 해당
 - 사용자 인터페이스를 지원하며 front-end라고도 함
 - 비즈니스 로직 계층
 - 기능 요구사항을 구현하는 곳으로 미들웨어라고도 함
 - 애플리케이션 로직을 실행하고 또 어떤 형태의 데이터가 필요하고 반환될 것인지를 결정
 - 데이터 계층
 - 데이터베이스에 접근해 데이터 처리와 관리를 하며 필요한 데이터를 제공
 - back-end라고도 함

아키텍처 스타일

- MVC 스타일

- 구성요소를 기능별 또는 특성별로 명확하게 분리해 유지보수를 쉽게 하고 프로그램의 확장성과 유연성을 높임
- 모델, 뷰, 제어의 세 부분으로 이루어짐
- 뷰와 모델을 독립적으로 분리해 변경에 대한 영향이 덜 미치도록 한 것이 장점
- 약한 결합으로 설계하여 서로 영향을 덜 미치기 때문에 구조 변경을 요청할 때 수정하기가 쉬움

①사용자는 제어에게 자료를 요청하거나 처리할 데이터를 전송

②제어는 이를 처리하기 위해 모델을 호출

③모델은 DBMS를 이용해 데이터베이스의 자료를 수정하고 그 결과를 제어에게 반환

④제어는 사용자가 요청한 결과를 모델로부터 가져와 뷰를 호출하고 결과를 전달

⑤뷰는 처리 결과를 여러 형태(테이블, 그래프, 액셀 등)로 사용자에게 공개

아키텍처 설계

아키텍처 스타일

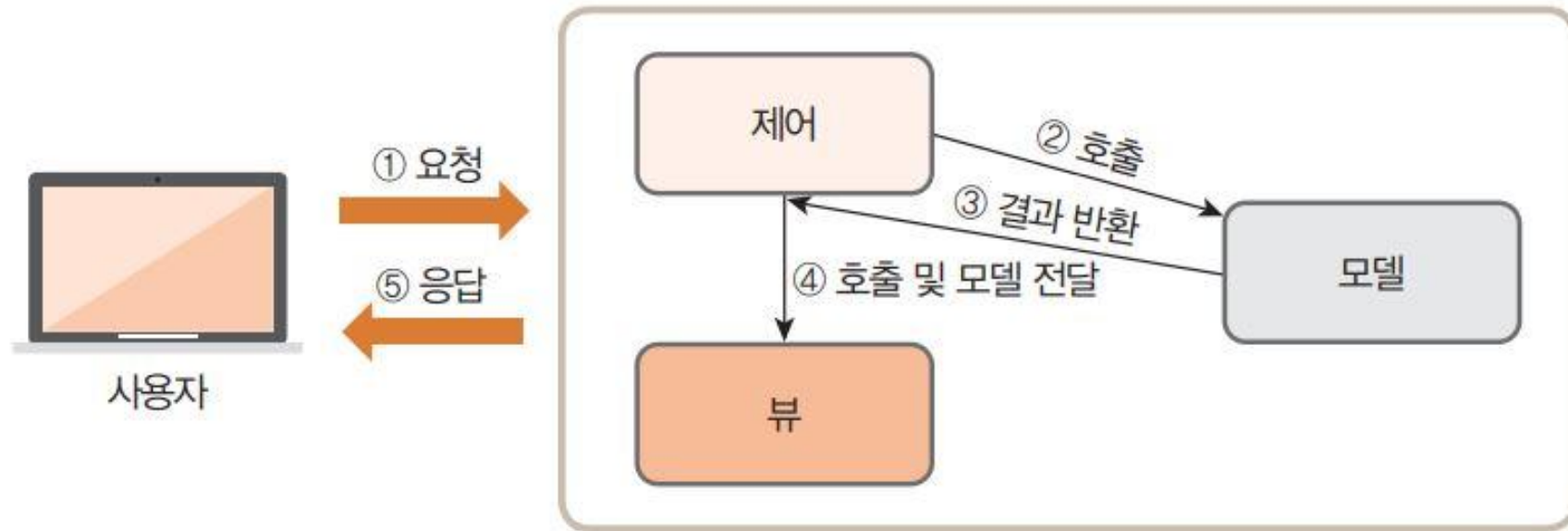


그림 6-8 MVC 스타일

아키텍처 스타일

• 데이터 흐름 스타일

- 서브시스템이 하나의 데이터를 입력으로 받아서 처리한 후 그 결과를 다음 서브시스템으로 넘겨주는 과정을 반복함
 - 일반적으로 데이터를 변환하는 시스템에서 주로 사용하며, 전체 변환 작업은 독립적인 단계로 나뉠 수 있음
 - 파이프와 필터를 조합해 만드는 아키텍처에 적합하고 사용자의 개입 없이 데이터 흐름이 전환되는 경우에 사용
 - 필터 또는 파이프 단위로 나누어 개발할 수 있기 때문에 동시에 개발이 가능하다는 것이 장점
-
- 필터: 데이터 스트림을 하나 이상 입력받아 처리(변환)한 후 데이터 스트림 하나를 출력, 하나의 필터는 여러 포트로 데이터를 보낼 수 있음
 - 파이프: 필터를 거쳐 생성된 데이터 스트림 하나를 다른 필터의 입력에 연결

아키텍처 스타일

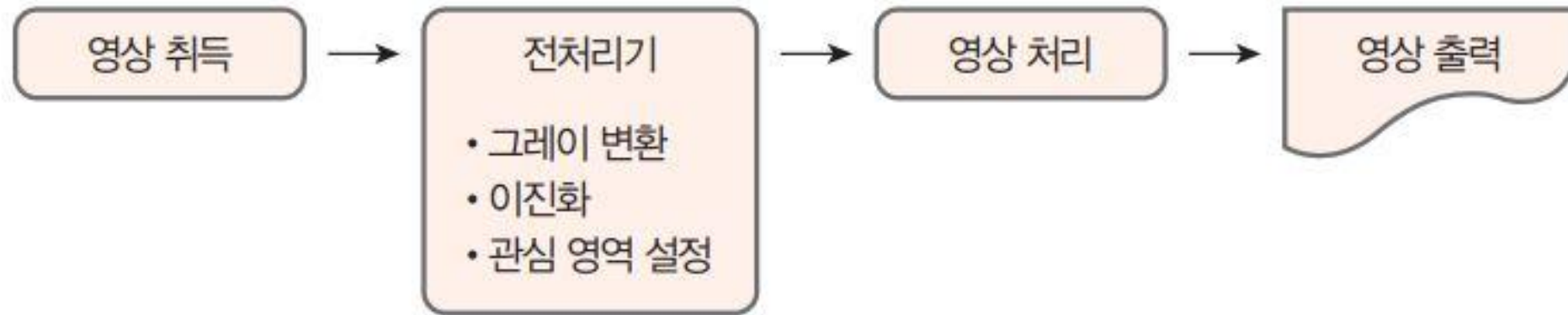


그림 6-9 데이터 흐름 스타일의 예: 이미지 프로세싱 과정

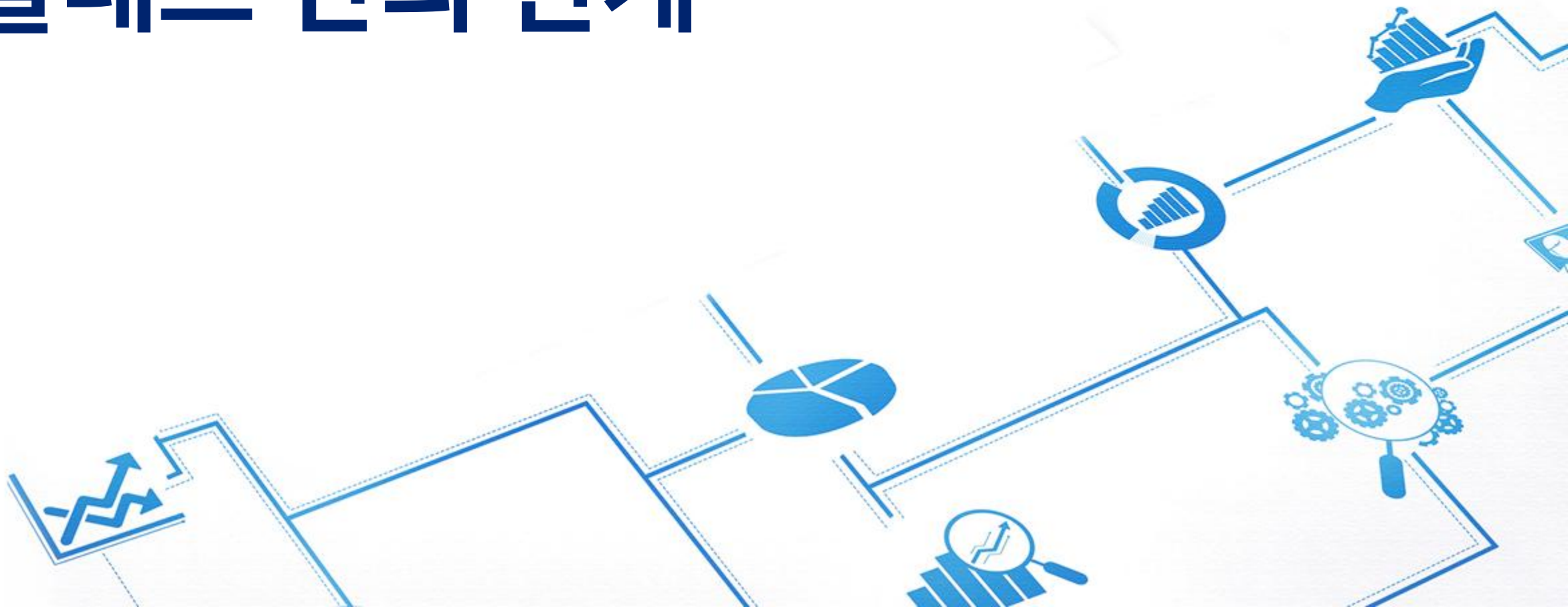
아키텍처 스타일

- 아키텍처 스타일의 장점
 - 개발 기간 단축 및 고품질의 소프트웨어 생산
 - 시행착오를 줄임으로써 개발 기간을 많이 단축 할 수 있고 고품질의 소프트웨어를 만들 수 있음
 - 수월한 의사소통
 - 아키텍처에 익숙한 개발자끼리 공통된 아키텍처를 공유함으로써 의사소통이 수월해짐
 - 용이한 유지보수
 - 개발에 참여하지 않은 사람이라도 비즈니스 로직만 잘 이해하고 있으면 유지 보수의 어려움을 많이 줄일 수 있음

아키텍처 스타일

- 아키텍처 스타일의 장점
 - 검증된 아키텍처
 - 사용을 통해 이미 검증된 아키텍처 스타일이므로 안정적으로 개발할 수 있음
 - 구축 전 시뮬레이션 가능
 - 이미 알려진 아키텍처 스타일을 이용하면 시스템을 개발하기 전에 시스템 특성을 시뮬레이션해볼 수 있음
 - 기존 시스템에 대한 빠른 이해
 - 기존 시스템이 어떤 아키텍처 스타일로 개발되었는지 알면 그 시스템을 더 빨리 이해할 수 있음

클래스 간의 관계



클래스 간의 관계

연관 관계

- 양방향 연관 관계

- 단순한 연관 관계로 '교수는 학생에게 수업하고, 학생은 교수에게 수업을 받는다'로 해석

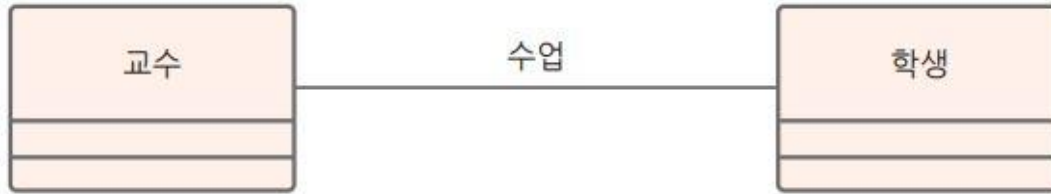


그림 6-10 연관 관계

- 역할이 부여된 연관 관계

- 연관 관계에서 각자에게 역할을 부여할 수 있음
- 교수의 역할은 '강의자'라 할 수 있고 학생의 역할은 '수강자'

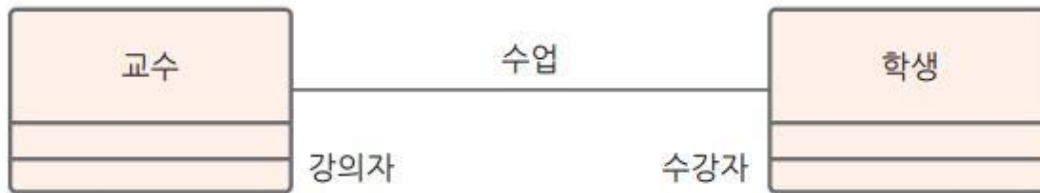


그림 6-11 역할이 부여된 연관 관계

클래스 간의 관계

연관 관계

• 다중 연관 관계

- 다중 관계는 다양하게 나올 수 있으며 선 위에 다중성을 표기해 나타냄
- 일 대 다(1 : n) 관계의 예는 교수 1명이 여러 명의 학생을 가르치는 관계라 할 수 있음
- 4명의 교수가 다수의 학생을 가르칠 경우도 있음
- 다 대 다 관계는 실제로 구현하기가 어려우므로 일 대 다 관계를 변환해 사용하는 것이 좋음

표 6-2 다중성 표기

표기	의미
1	오직 하나. 즉 1
0..*(또는 *)	다수(0 포함). 즉 0(없거나) 또는 1 이상
1..*	다수(0 제외). 즉 1 이상
0..1	0(없거나) 아니면 1. 즉 0 또는 1
3..6	3에서 6까지 중 하나. 즉 3 또는 4 또는 5 또는 6
2..4..6	2 또는 4 또는 6
1..4..6	1 또는 4에서 6까지 중 하나. 즉 1 또는 4 또는 5 또는 6

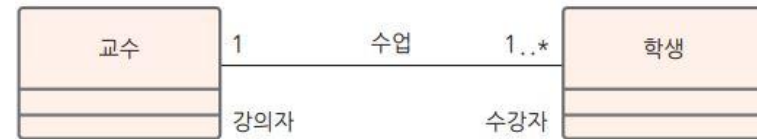


그림 6-12 일 대 다(1 : n) 관계



그림 6-13 4 대 다(4 : n) 관계



그림 6-14 다 대 다(n:n) 관계

클래스 간의 관계

연관 관계

- 단방향 연관 관계
 - 두 클래스가 서로 아는 관계가 아니고 한쪽만 아는 관계를 나타냄
 - 두 클래스 간의 연결선이 방향성을 나타내는 화살표가 있는 직선 실선



그림 6-15 단방향 연관 관계

클래스 간의 관계

연관 관계

- 연관 클래스

- 연관 관계를 더 구체적으로 나타내고 싶을 때 클래스를 추가해 사용하는 것
- 점선을 사용해 나타내며, 일반 클래스처럼 다른 클래스와도 연관 관계를 맺을 수 있

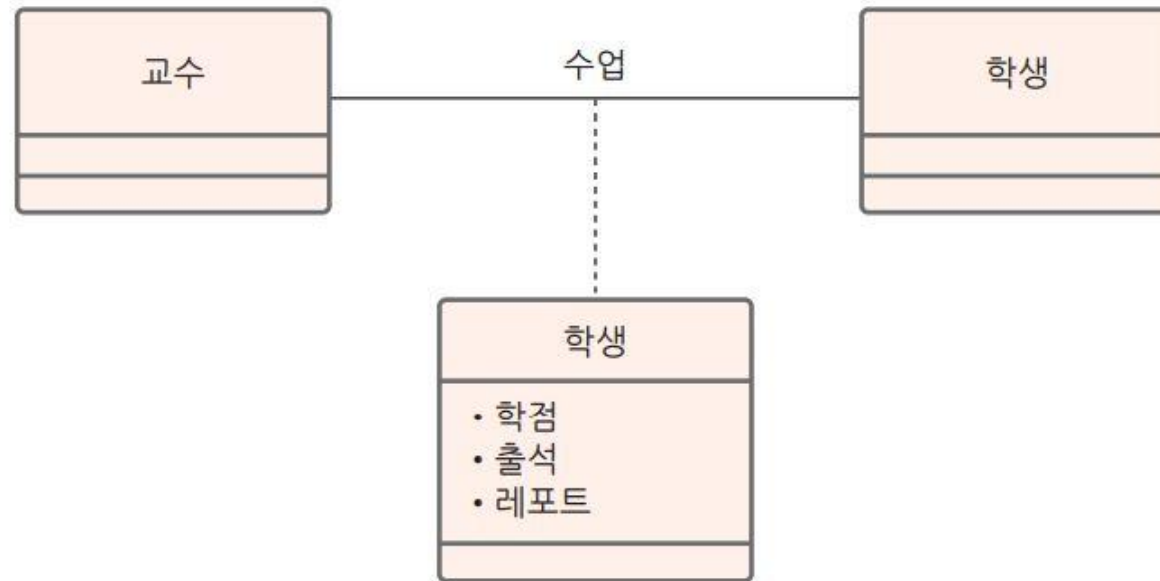


그림 6-16 연관 클래스

클래스 간의 관계

일반화 관계

- 일반화와 상속

- '일반적'의 사전적 의미: 일부에 한정되지 아니하고 전체에 걸치는 것 , 보편적이고 상식적인 것
- 클래스에서 일반화: 공통점을 가지고 있는 여러 개의 클래스를 묶어서 새로운 클래스를 만들고 공통적인 이름을 붙인 것
- 일반화 관계는 상속 구조이며 하위 클래스는 상위 클래스의 모든 것(속성, 메서드)을 상속받아 사용
- 아래에서 위로 추상화되는 것을 일반화, 반대로 위에서 아래로 구체화하는 것을 특수화라고 함
- 개별 클래스와 공통 클래스 사이에 'is a kind of' 관계가 성립되어야 함
 - is a kind of 관계: '사각형은 도형의 일종이다', '원은 도형의 일종이다'

클래스 간의 관계

일반화 관계

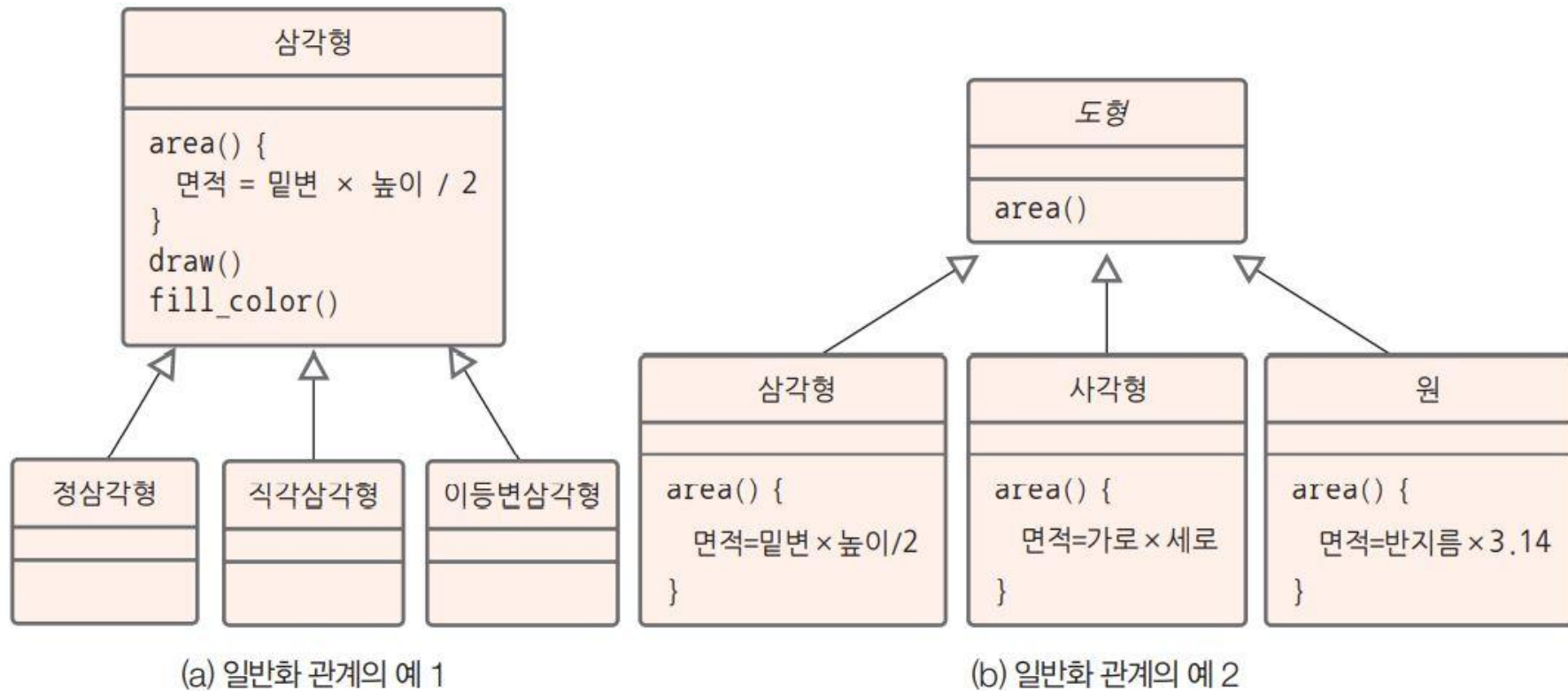


그림 6-17 일반화(상속) 관계

클래스 간의 관계

집합 관계

- 집합 관계

- '상위 클래스가 하위 클래스로 구성될 때의 관계로 'is composed of'가 성립되어야 함
 - is composed of 관계: '컴퓨터는 모니터, 본체, 키보드로 이루어졌다'라고 할 때 말이 되는 관계
- 모든 객체가 별개의 생명주기를 가지고 있으며 각각 독립적으로 동작하기 때문에 약한 결합 관계
- 집합 관계에서 전체(컴퓨터)와 부분(모니터, 본체, 키보드)의 연결은 직선으로 하되 머리 부분은 속이 비어 있는 마름모 모양

클래스 간의 관계

집합 관계

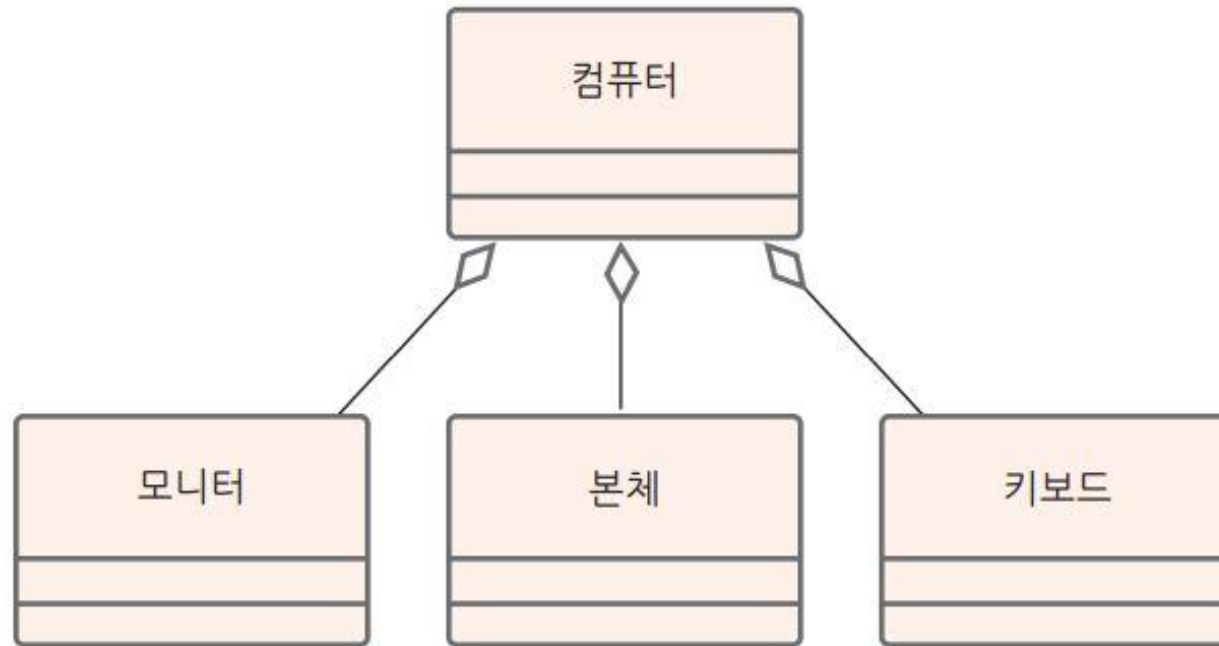


그림 6-18 집합 관계의 예

클래스 간의 관계

합성 관계

- 합성 관계

- 집합 관계와 많은 부분이 유사하지만 전체 객체에 완전히 종속되어 독립된 객체로 존재할 수 없다는 것이 다름
- 모든 객체가 같은 생명주기를 가지고 있으므로 각각 독립적으로 동작할 수 없는 강한 결합 관계
- 전체(노트북)와 부분(모니터, 본체, 키보드)의 연결은 직선으로 하되 머리 부분은 속이 채워진 마름모 모양

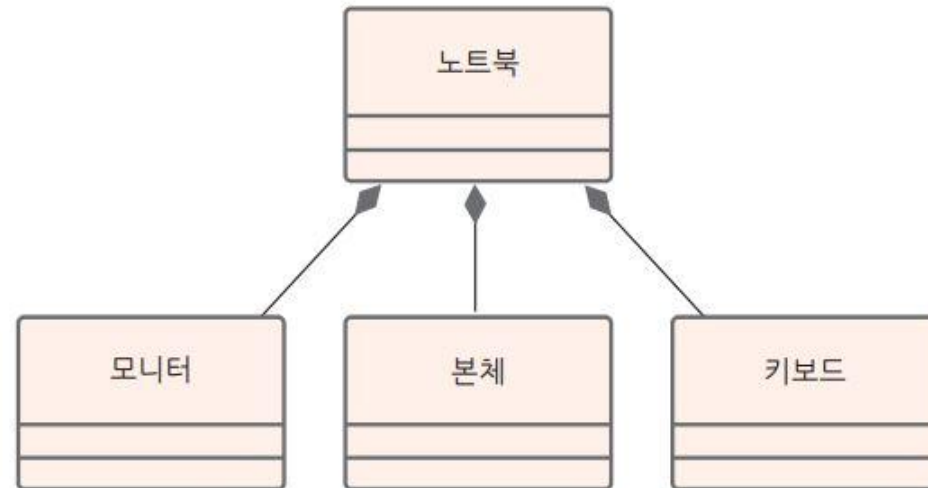


그림 6-19 합성 관계의 예

클래스 간의 관계

의존 관계

- 연관 관계와 의존 관계의 차이점
 - 서로 상대의 클래스를 사용(참조)할 때의 관계로 클래스 A의 변화는 클래스 B의 변화로 연결되는 점에서 연관 관계와 유사
 - 연관 관계는 상대 클래스를 인식하기 위해 멤버 변수를 가짐
 - 클래스 A인 Repeater 클래스가 클래스 B인 Scan 클래스를 멤버 변수로 가지고 있음

클래스 간의 관계

의존 관계

코드 6-1 Repeater 클래스

```
public class Repeater {  
    // Scan 클래스를 멤버 변수로 사용  
    public Scan scan = new Scan();  
    public void repeat() {  
        int count = 0;  
        int max = scan.returnInt();  
        while (count < max) {  
            System.out.println("안녕하세요");  
            count++;  
        }  
    }  
}
```

코드 6-2 Scan 클래스

```
import java.util.Scanner;  
public class Scan {  
    Scanner sc = new Scanner(System.in);  
    int returnInt() {  
        System.out.print("숫자를 입력해주세요 : ");  
        return sc.nextInt();  
    }  
}
```

코드 6-3 Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        Repeater repeater = new Repeater();  
        repeater.repeat();  
    }  
}
```

실행 결과

```
숫자를 입력해주세요 : 5  
안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요
```


클래스 간의 관계

의존 관계

- 연관 관계와 의존 관계의 차이점
 - 의존 관계는 상대의 메서드를 가지고 있음
 - 예시) 클래스 A의 메서드가 클래스 B의 객체를 인자로 받아 그 메서드를 사용할 경우
 - 색 글자로 표시된 부분을 보면 Scan 클래스를 인자(파라미터)로 받아 사용

코드 6-4 Repeater 클래스

```
public class Repeater {  
    // Scan 클래스를 인자로 사용  
    // Main함수에서 new Scan() 필요.  
    public void repeat(Scan scan) {  
        // Repeater 클래스와 동일  
    }  
}
```

코드 6-5 Main 클래스

```
public class Main {  
    public static void main(String args[]) {  
        Repeater repeater = new Repeater();  
        repeater.repeat(new Scan());  
    }  
}
```

클래스 간의 관계

의존 관계

- 연관 관계와 의존 관계의 차이점
 - 의존 관계는 상대의 메서드를 가지고 있음
 - 예시) 클래스 A인 Repeater 클래스의 메서드인 repeat() 내부에서 클래스 B의 객체를 생성해 메서드를 사용할 경우
 - 클래스 A인 Repeater 클래스는 클래스 B인 Scan 클래스를 지역 변수로 사용

코드 6-6 Repeater 클래스

```
public class Repeater {  
    //Scan 클래스를 지역 변수로 선언하여 사용  
    public void repeat() {  
        Scan scan = new Scan();  
        int count = 0;  
        int max = scan.returnInt();  
        while (count < max) {  
            System.out.println("안녕하세요");  
            count++;  
        }  
    }  
}
```

클래스 간의 관계

의존 관계

- 실체화 관계

- 특정 클래스에 기능을 추가하기 위해 메서드를 넣을때 하위 클래스가 상속을 무조건 받아야 하는 사태가 발생
- 인터페이스 클래스는 메서드의 공통 특성을 묶어 새로운 인터페이스 클래스를 만들고 공통의 이름을 붙임
- 이 클래스는 변수를 정의할 수 없고 추상 메서드를 가지며 이 메서드에 대한 구체적인 실현은 하위 클래스에서 구현
- 스테레오타입으로 <<>>를 사용하고 하위 클래스와의 연관은 일반화 관계와 다르게 점선으로 나타냄
- 화살표의 머리 부분은 하위 클래스에서 상위 클래스로 향하며 모양은 속이 빈 삼각형

클래스 간의 관계

의존 관계

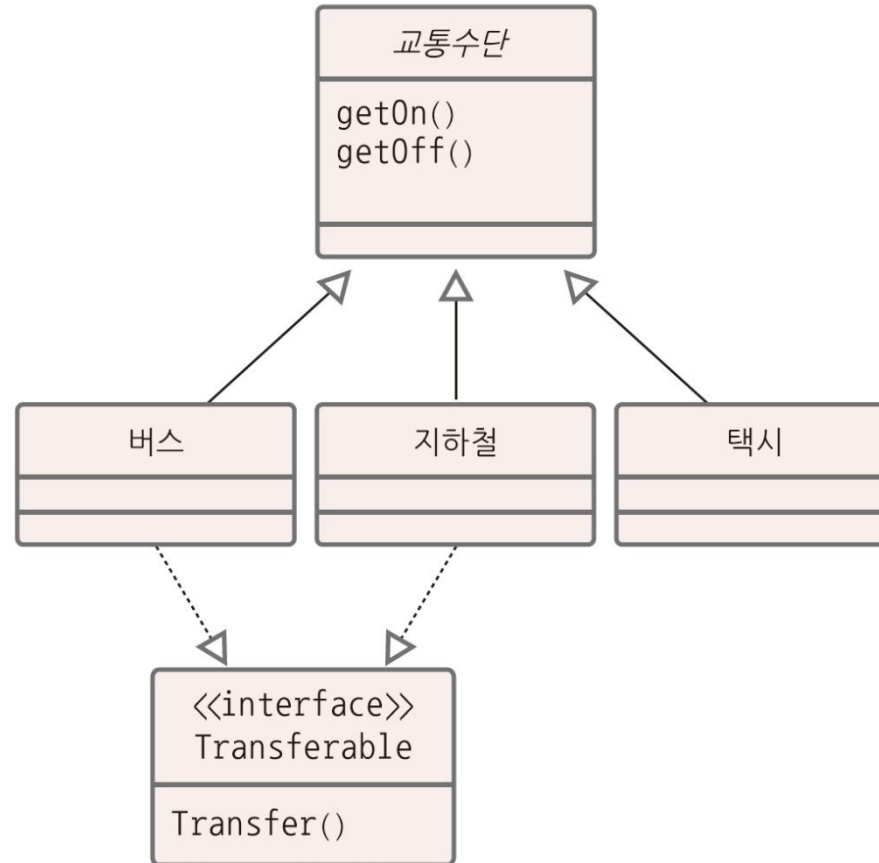
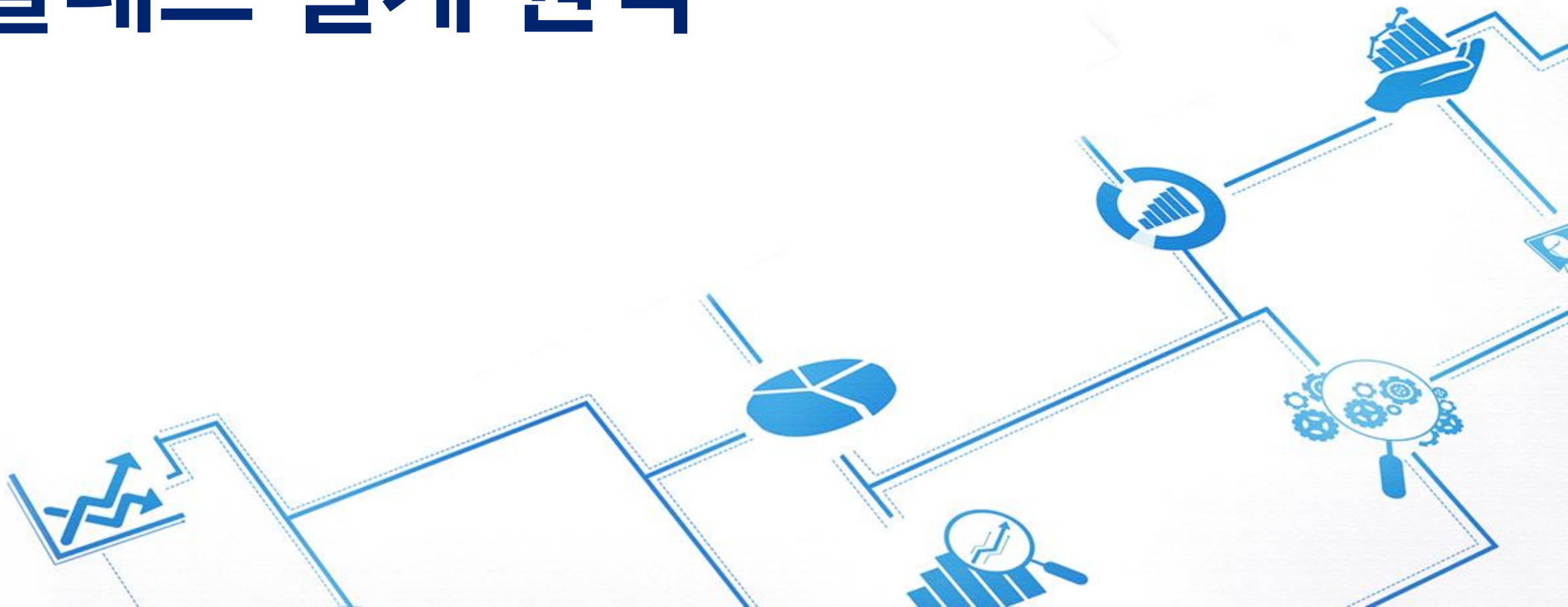


그림 6-20 실체화의 예

클래스 설계 원칙



클래스 설계 원칙

단일 설계 원칙(SRP)

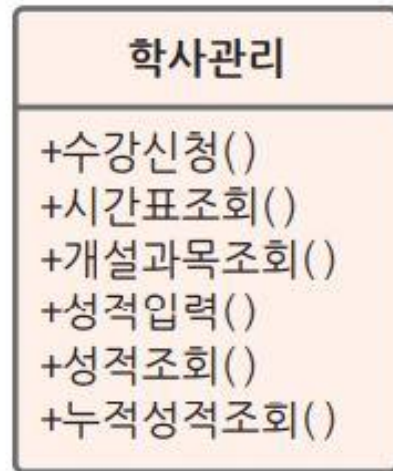
- 단일 책임 원칙

- 로버트 마틴: 책임의 의미는 변경하는 이유
- 단일 책임의 원칙은 클래스를 설계할 때 변경하는 이유가 하나가 되도록 해야 한다는 것
- 모든 클래스는 한 가지 책임, 즉 변경의 이유를 오직 하나만 갖도록 설계하고 클래스는 그 책임을 완전히 캡슐화하는 것
- 클래스에 2개의 책임이 존재한다면 2개의 클래스로 분리해 '변경해야 하는 이유가 오직 하나 가 되도록' 설계하는 것이 바람직

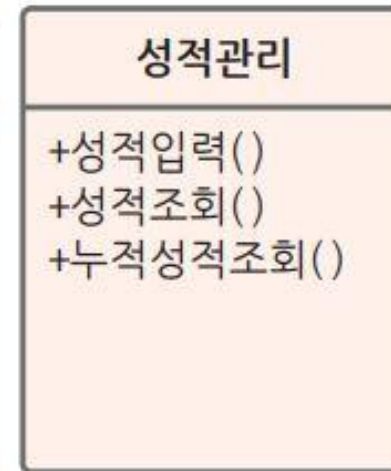
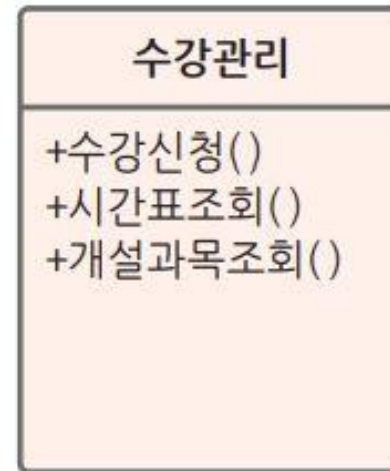
단일 책임 원칙: 클래스를 변경해야 하는 이유는 단 하나여야 한다.

클래스 설계 원칙

단일 설계 원칙(SRP)



(a) 적용 전



(b) 적용 후

그림 6-21 단일 책임 원칙의 적용 예

클래스 설계 원칙

개발 폐쇄 원칙(OCP)

- 개방 폐쇄 원칙
 - 어떤 것은 개방하고 어떤 것은 폐쇄하는 것을 말함
- Video_Palyer 클래스의 예시: 원칙 위반
 - 처음에는 MP4만 지원하던 Video_Palyer 클래스에 지원가능한 파일 형식이 늘어날때마다 클래스를 수정
 - 개방 폐쇄 원칙을 따르지 않으면 지원 가능한 파일 형식이 늘어날 때마다 계속 클래스를 수정해야 함

개방 폐쇄 원칙: 변경에는 닫혀 있어야 하고, 확장에는 열려 있어야 한다.

클래스 설계 원칙

개발 폐쇄 원칙(OCP)

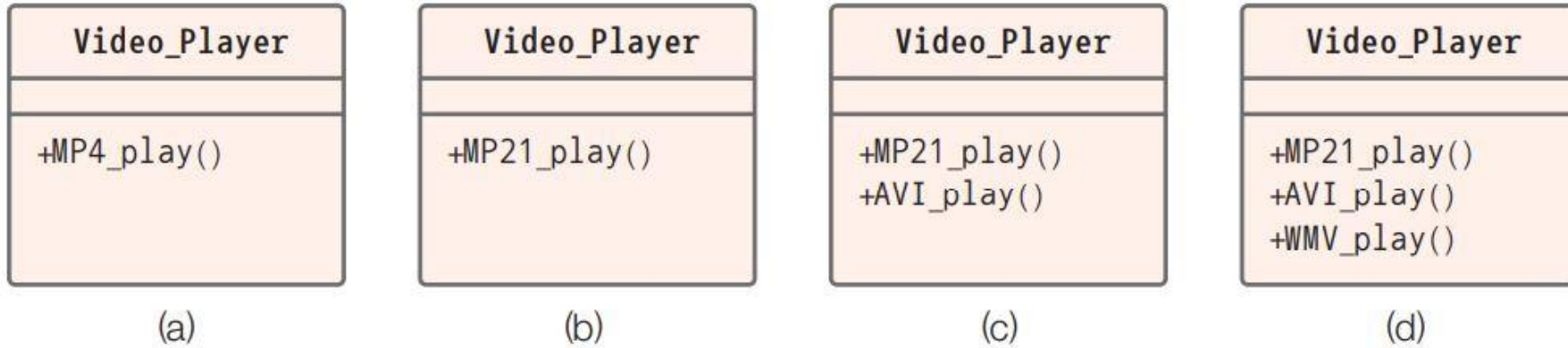


그림 6-22 개방 폐쇄 원칙을 따르지 않은 클래스 설계

클래스 설계 원칙

개발 폐쇄 원칙(OCP)

- Video_Palyer 클래스의 예시: 원칙 적용
 - 계속해서 바뀌는 것이 무엇인지를 찾고 그것들을 모아 상속 구조로 만들어 상위 클래스에 배치
 - 하위 클래스에는 코덱의 종류를 배치해 계속 추가할 수 있도록 함
 - Video_Player 클래스에서 이들을 호출해 사용하도록 설계
- 개방 폐쇄 원칙의 장점
 - 변경에 닫혀 있는 설계: 계속된 추가 사항이 있어도 그들끼리만 추가할 수 있도록 설계
 - 확장에 열려 있는 설계: 새로운 클래스를 쉽게 추가할 수 있는 구조로 설계

클래스 설계 원칙

개발 폐쇄 원칙(OCP)

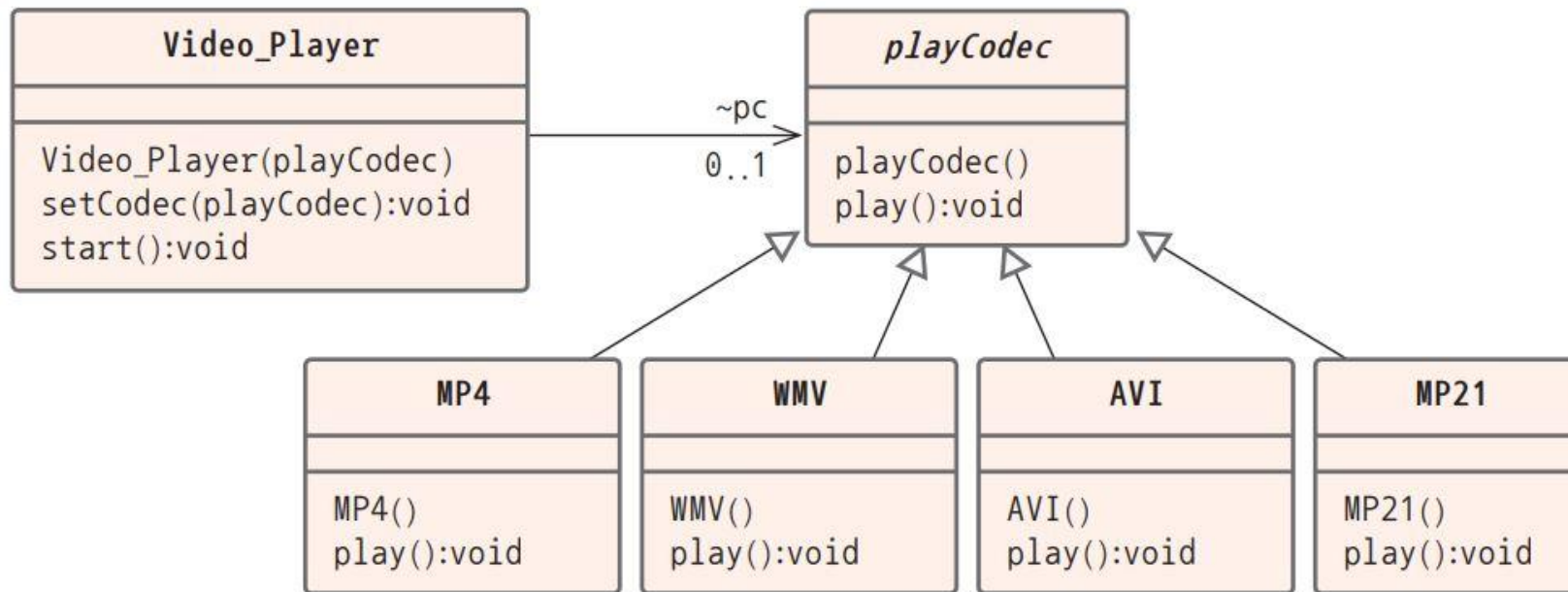


그림 6-23 개방 폐쇄 원칙을 따른 클래스 설계

클래스 설계 원칙

리스코프 교체 원칙(LSP)

- 리스코프 교체 원칙

- 상위 클래스의 객체가 들어갈 자리에 하위 클래스의 객체를 넣어도 문제없이 잘 작동해야 함
- 리스코프 교체 원칙은 상속과 재정의의 중요성을 강조함
- LSP를 지키는 클래스 설계를 위해서는 상속 구조를 만들 때 상위 클래스 추상 클래스와 추상 메서드여야 함
- 재정의할 때는 피터 코드의 상속 규칙에 맞게 사용해야 함
- 피터 코드의 상속 규칙: '하위 클래스는 상위 클래스의 책임을 무시하거나 재정의하지 않고 확장만 수행한다'

리스코프 교체 원칙: 상위 클래스의 객체는 언제나 자신의 하위 클래스의 객체로 교체할 수 있어야 한다.

클래스 설계 원칙

리스코프 교체 원칙(LSP)

- 리스코프 교체 원칙: 대학의 강사료 계산 프로그램의 예시

코드 6-7 LSP 위반 Lecturer_Before 클래스

```
public class Lecturer_Before {  
    public Lecturer() {  
        System.out.println("[리스코프 교체 원칙 위반]");  
    }  
    public String lecturer;  
    public int charge = 90000;  
    public void Lecturer() {  
        System.out.println("일반 강사");  
        System.out.println("강사료: " + charge + "원\n");  
    }  
}
```

코드 6-8 LecturerNight 클래스

```
public class LecturerNight extends Lecturer_Before {  
    public LecturerNight() {  
        lecturer = "야간 강사";  
    }  
    public void Lecturer() {  
        System.out.println(lecturer);  
        System.out.println("강사료: " + charge*1.5 + "원\n");  
    }  
}
```

코드 6-9 LecturerAtGS 클래스

```
public class LecturerAtGS extends Lecturer_Before {  
    public LecturerAtGS() {  
        lecturer = "대학원 강사";  
    }  
    public void Lecturer() {  
        System.out.println(lecturer);  
        System.out.println("강사료: " + charge*2 + "원\n");  
    }  
}
```

코드 6-10 LecturerWithWork 클래스

```
public class LecturerWithWork extends Lecturer_Before {  
    public LecturerWithWork() {  
        lecturer = "직업이 있는 강사";  
    }  
    public void Lecturer() {  
        System.out.println(lecturer);  
        System.out.println("강사료: " + charge*1/2 + "원\n");  
    }  
}
```

클래스 설계 원칙

리스코프 교체 원칙(LSP)

- 리스코프 교체 원칙: 대학의 강사료 계산 프로그램의 예시
 - 이 클래스 다이어그램은 Lecturer()를 하위 클래스에서 재정의해 사용함
 - 상위 클래스의 Lecturer()를 대학원 시간 강사의 Lecturer()로 대체하면 문제가 발생
 - 일반 강사의 강사료는 90,000, 대학원 시간 강사의 강사료는 180,000원으로 대체 시 일반 강사가 180,000를 받게됨
 - 재정의를 무분별하게 사용했기 때문에 오류가 발생

클래스 설계 원칙

리스코프 교체 원칙(LSP)

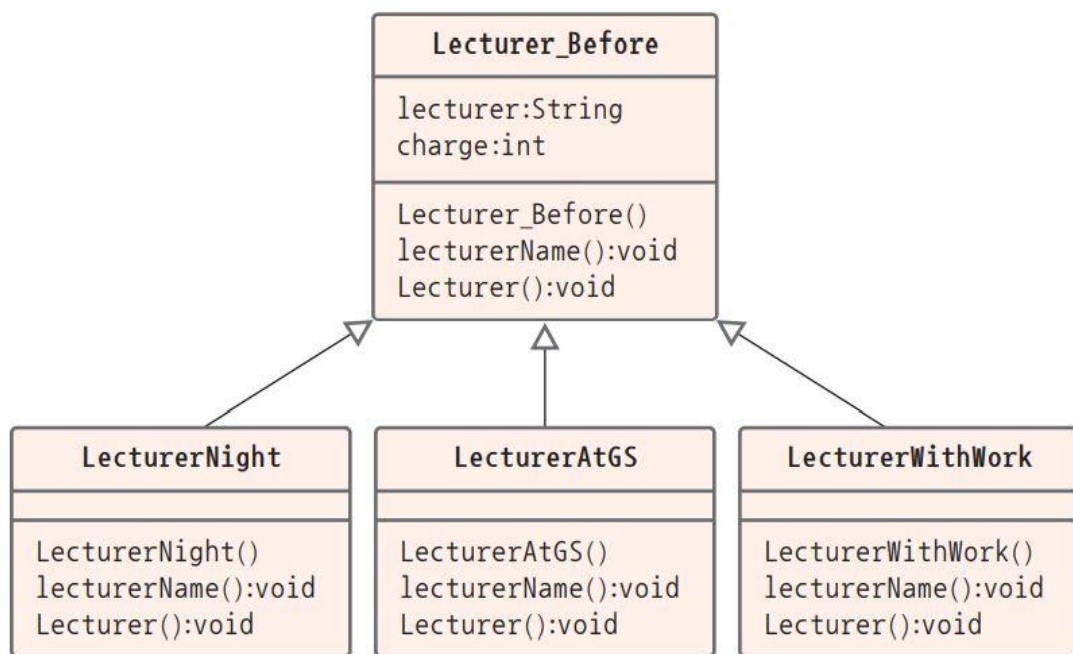


그림 6-24 리스코프 교체 원칙(LSP)을 위반한 클래스 다이어그램

코드 6-11 Main 클래스

```
public class Main {
    public static void main(String args[]) {
        LecturerAtGS lect01 = new LecturerAtGS();
        lect01.Lecturer();

        LecturerNight lect02 = new LecturerNight();
        lect02.Lecturer();

        LecturerWithWork lect03 = new LecturerWithWork();
        lect03.Lecturer();

        Lecturer_Before lect04 = new Lecturer_Before();
        lect04.Lecturer();
    }
}
```

실행 결과

[리스코프 교체 원칙 위반]

대학원 강사

강사료: 180000원

[리스코프 교체 원칙 위반]

야간 강사

강사료: 135000원

[리스코프 교체 원칙 위반]

직업이 있는 강사

강사료: 45000원

[리스코프 교체 원칙 위반]

일반 강사

강사료: 90000원

클래스 설계 원칙

리스코프 교체 원칙(LSP)

- 리스코프 교체 원칙: 대학의 강사료 계산 프로그램의 예시
 - 재정의는 상위 클래스의 메서드가 추상 메서드일 경우에만 사용

코드 6-12 LSP를 적용한 Lecturer 클래스

```
public abstract class Lecturer {  
    public Lecturer() {  
        System.out.println("[리스코프 교체의 원칙]\n");  
    }  
    public String lecturer;  
    public int charge = 90000;  
    public void Lecturer() {  
    }  
}
```

코드 6-13 LSP를 적용한 LecturerNormal 클래스

```
public class LecturerNormal extends Lecturer {  
    public LecturerNormal() {  
        lecturer = "일반 강사";  
    }  
    public void Lecturer() {  
        System.out.println(lecturer);  
        System.out.println("강사료 :" + charge + "원\n");  
    }  
}
```


클래스 설계 원칙

의존 관계 역전 원칙(DIP)

- 의존 관계 역전 원칙

- 객체지향 설계에서 서로의 관계가 의존적이라는 것은 결합도가 높다는 것이고 이는 클래스 설계에서 좋은 관계가 아님
- 의존 관계 역전 원칙에서는 구체 클래스에 의존하도록 클래스 설계를 하지 않고 자주 바뀌는(추가/삭제) 클래스를 상속 구조로 만들어 추상 클래스에 의존하도록 설계
- '고수준 구성 요소가 저수준 구성 요소에 의존하면 안된다'는 의미

의존 관계 역전 원칙: 클라이언트는 구체 클래스가 아닌 추상 클래스(인터페이스)에 의존해야 한다

클래스 설계 원칙

의존 관계 역전 원칙(DIP)

- **GameServer 클래스의 예시: 원칙 위반**
 - 조건문을 추가하여 조건의 변화(추가/삭제)에 따라 수정할 수밖에 없다는 것을 의미하므로 OCP(개방 폐쇄 원칙) 위반
 - 구체 클래스인 GameServer에서 직접 생성하면 GameServer 클래스는 생성된 모든 게임 클래스에 직접 의존
 - 각 게임 클래스가 업데이트 될 경우 GameServer 클래스는 객체선언부터 조건문까지 모두 변경해야함

클래스 설계 원칙

의존 관계 역전 원칙(DIP)

코드 6-14 DIP 적용 전의 GameServer 클래스

```
public class GameServer {
    public String title;
    public String version;
    public SuperMario supermario;
    public Tetris tetris;
    public LOL lol;
    public Racing racing;
    public void Game_Play(String game) {
        choiceGame(game);
        Start();
    }
    public void Start() {
        System.out.println("게임명 : " + title);
        System.out.println("버전 : " + version);
        System.out.println(title + "을(를) 시작합니다.\n");
    }
    public void choiceGame(String game) {
        if (game.equals("supermario")) {
            supermario = new SuperMario();
            title = supermario.returnTitle();
            version = supermario.returnVersion();
        } else if (game.equals("tetris")) {
            tetris = new Tetris();
            title = tetris.returnTitle();
            version = tetris.returnVersion();
        } else if (game.equals("lol")) {
            lol = new LOL();
            title = lol.returnTitle();
            version = lol.returnVersion();
        } else if (game.equals("racing")) {
            racing = new Racing();
            title = racing.returnTitle();
            version = racing.returnVersion();
        } else
            System.out.println("지원하지 않는 게임입니다.");
    }
}
```

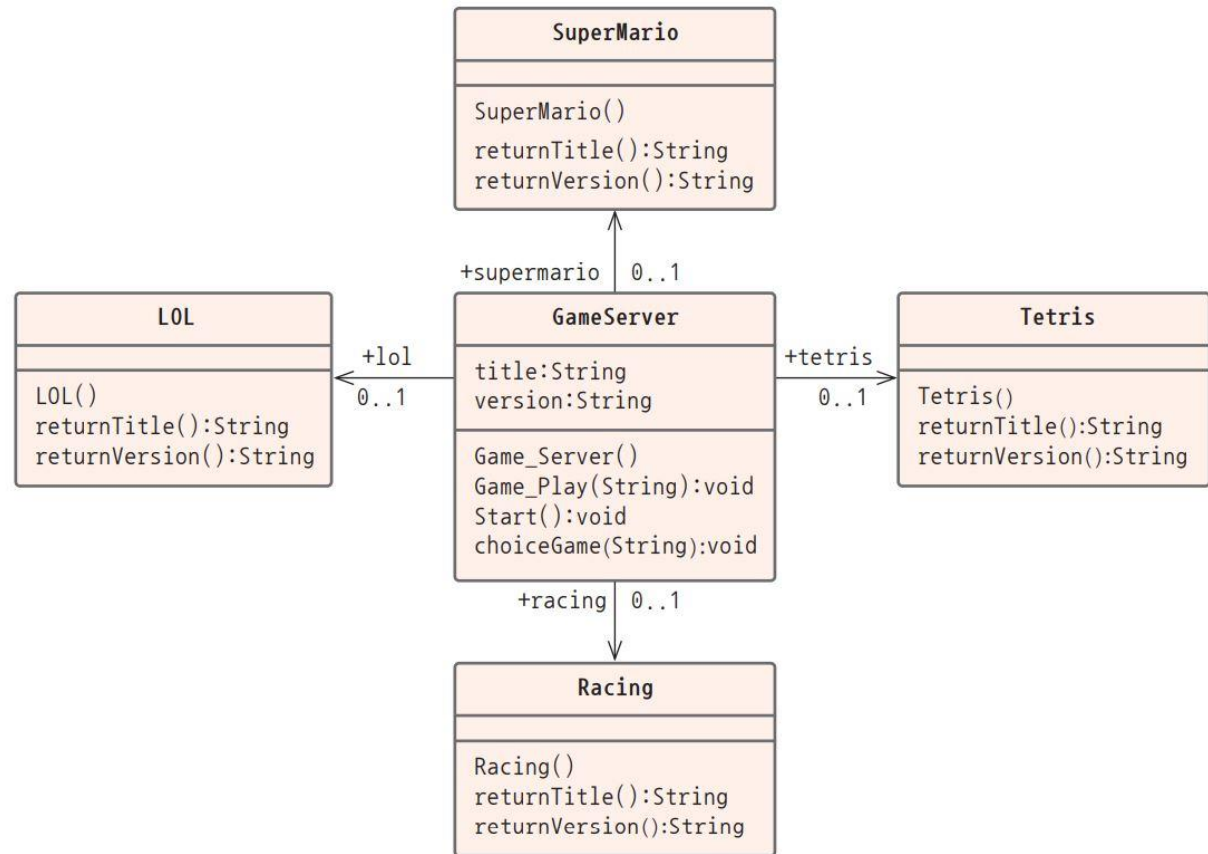


그림 6-25 의존 관계 역전 원칙(DIP) 사용 전 클래스 다이어그램

클래스 설계 원칙

의존 관계 역전 원칙(DIP)

- GameServer 클래스의 예시: 원칙 적용
 - 게임을 쉽게 추가/삭제할 수 있도록 상속 구조로 만듦
 - GameServer 클래스가 추상 클래스인 Games를 참조하도록 만듦
 - 클래스 다이어그램을 보면 GameServer가 Games라는 추상 클래스에 의존
 - 모든 게임 클래스도 추상 클래스인 Games에 의존

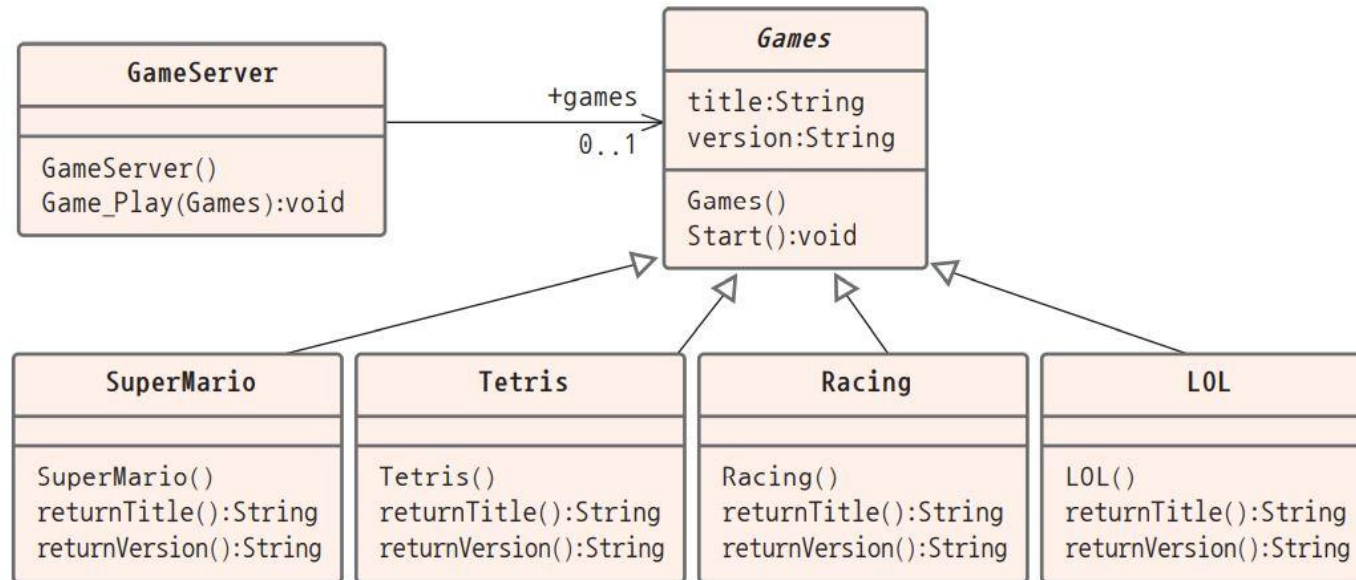


그림 6-26 의존 관계 역전 원칙(DIP)을 적용한 클래스 다이어그램

클래스 설계 원칙

의존 관계 역전 원칙(DIP)

- GameServer 클래스의 예시: 원칙 적용

코드 6-15 DIP를 적용한 GameServer 클래스

```
public class GameServer {  
    public Games games;  
    public void Game_Play(Games games) {  
        games.Start();  
    }  
}
```

코드 6-16 DIP를 적용한 Games 클래스

```
public abstract class Games {  
    public String title, version;  
  
    public void Start() {  
        System.out.println("게임명 : " + title);  
        System.out.println("버전 : " + version);  
        System.out.println(title + "을(를) 시작합니다.\n");  
    }  
}
```

클래스 설계 원칙

의존 관계 역전 원칙(DIP)

- GameServer 클래스의 예시: 원칙 적용

코드 6-17 LOL 클래스

```
public class LOL extends Games {
    public LOL() {
        title = returnTitle();
        version = returnVersion();
    }
    public String returnTitle() {
        return "League of Legends";
    }
    public String returnVersion() {
        return "v.2020";
    }
}
```

코드 6-18 Racing 클래스

```
public class Racing extends Games {
    public Racing() {
        title = returnTitle();
        version = returnVersion();
    }
    public String returnTitle() {
        return "Racing";
    }
    public String returnVersion() {
        return "v2.3";
    }
}
```

코드 6-21 Main 클래스

```
public class Main {
    public static void main (String args[]) {
        GameServer gameserver = new GameServer();
        gameserver.Game_Play(new SuperMario());
        gameserver.Game_Play(new Tetris());
        gameserver.Game_Play(new LOL());
        gameserver.Game_Play(new Racing());
    }
}
```

코드 6-19 SuperMario 클래스

```
public class SuperMario extends Games {
    public SuperMario() {
        title = returnTitle();
        version = returnVersion();
    }
    public String returnTitle() {
        return "Super Mario";
    }
    public String returnVersion() {
        return "v1.0";
    }
}
```

코드 6-20 Tetris 클래스

```
public class Tetris extends Games {
    public Tetris() {
        title = returnTitle();
        version = returnVersion();
    }
    public String returnTitle() {
        return "Tetris";
    }
    public String returnVersion() {
        return "v1.3";
    }
}
```

클래스 설계 원칙

인터페이스 분리 원칙(ISP)

- 인터페이스 분리 원칙
 - 다수의 클라이언트가 일반적인 인터페이스 하나를 같이 사용하는 것보다 각각의 클라이언트가 필요한 대로
 - 여러 개의 구체적인 인터페이스로 분리해 사용하는 것이 더 낫다는 의미
 - 무조건 모든 클라이언트에게 각각의 인터페이스를 제공하는 의미가 아님
 - 각각의 클라이언트가 필요로 하는 메서드군이 존재할 때 인터페이스를 분리
- 인터페이스 분리 원칙의 장점
 - 용도가 명확한 인터페이스를 제공할 수 있어 클래스에 필요한 메서드만 선언할 수 있음
 - 시스템의 내부 의존성을 낮춰(낮은 결합) 리팩토링을 쉽게 해 재사용성을 높일 수 있음

인터페이스 분리 원칙: 클라이언트는 자신이 사용하지 않는 메서드와 의존 관계를 맺으면 안 된다.

클래스 설계 원칙

인터페이스 분리 원칙(ISP)

- 학사관리 클래스의 예시: 원칙 위반

- 학사관리 클래스에 많은 메서드가 있고 이 메서드를 학생, 교수, 직원이 사용
- 학생이 사용하는 것은 수강신청()과 성적조회()뿐인데 사용하지 않는 다른 메서드와도 연관 관계를 맺고 있음
- 직원과 교수도 마찬가지로 사용하지 않는 메서드와 연관 관계를 맺고 있음

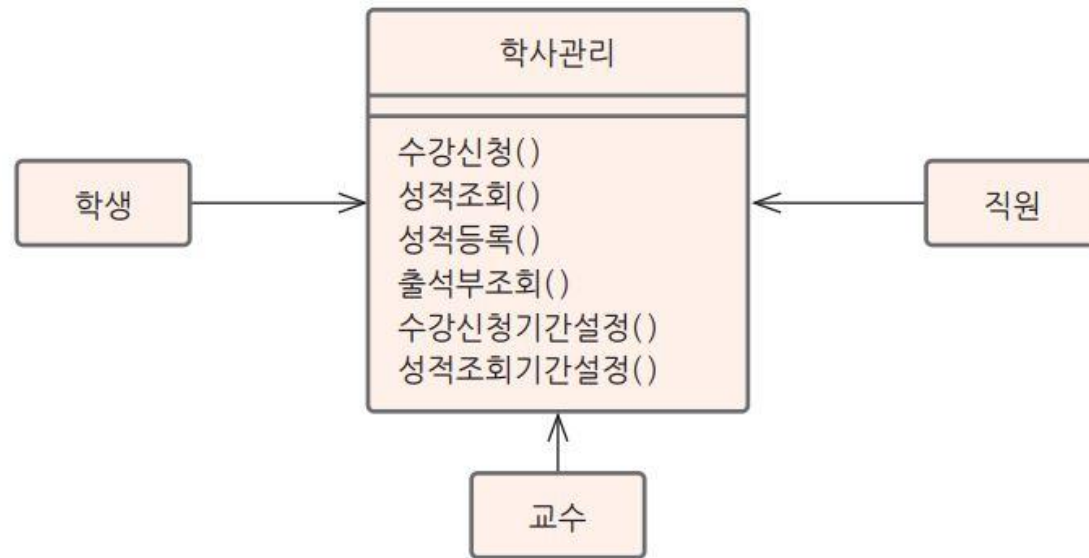


그림 6-27 인터페이스 분리 원칙에 위배되는 클래스 다이어그램

클래스 설계 원칙

인터페이스 분리 원칙(ISP)

- 학사관리 클래스의 예시: 원칙 적용
 - 클래스 다이어그램에서는 사용자(학생, 직원, 교수)별로 인터페이스를 분리
 - 각 사용자는 오직 자신이 필요로 하는 메서드와 연관되어 있고 다른 메서드와는 관계가 없음
 - 학생 인터페이스의 수강신청(), 성적조회()의 구현은 학사관리 클래스에 그대로 남아 있음
 - '단일 책임 원칙'과 혼동해서는 안됨

클래스 설계 원칙

인터페이스 분리 원칙(ISP)

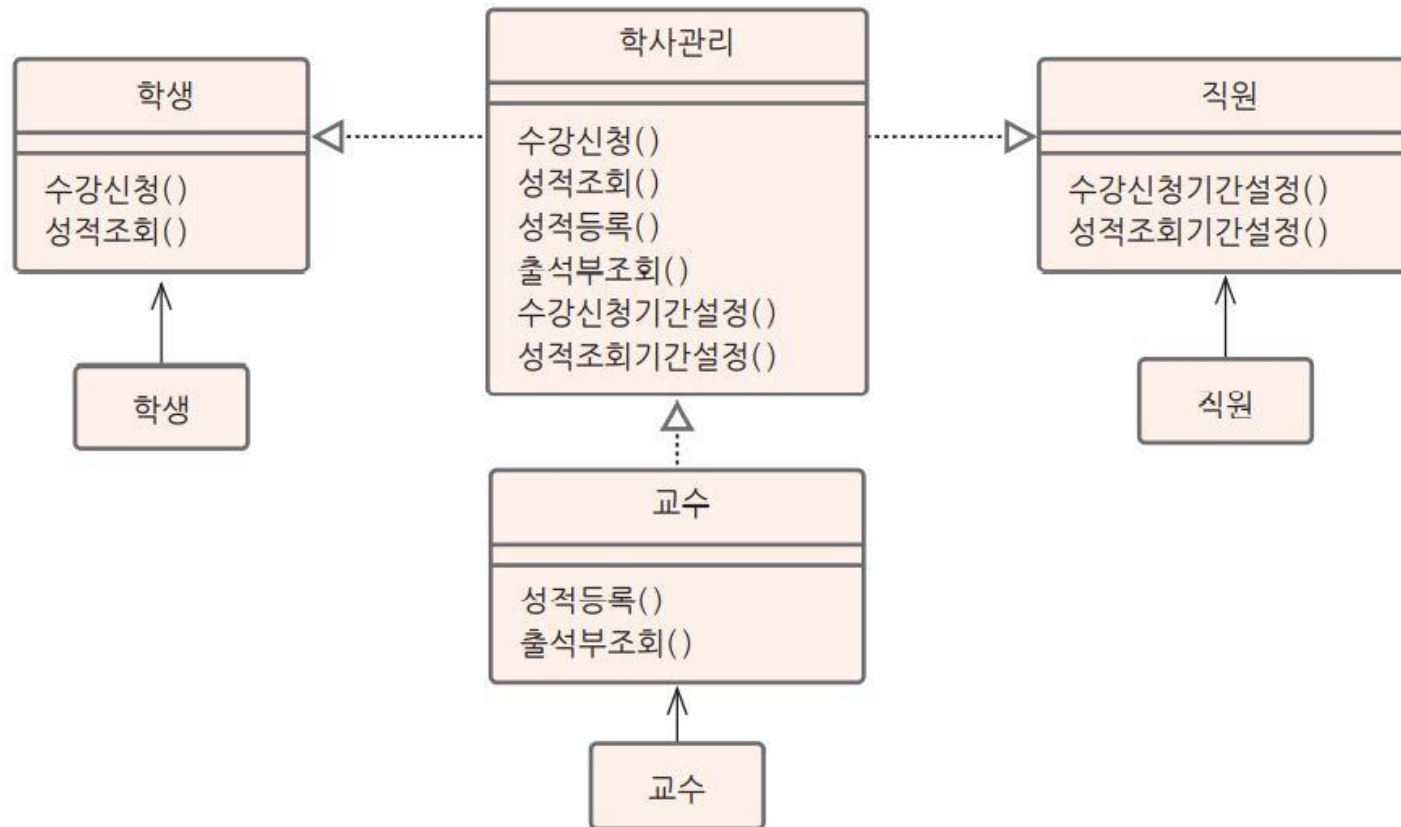


그림 6-28 인터페이스 설계 원칙을 적용한 클래스 다이어그램

END

